National Research University Higher School of Economics
Faculty of Computer Science
Programme ‘Master of Data Science’

MASTER’S THESIS

The Use of Machine Learning for Predictive Analytics of Incoming and Outgoing Flow of Goods in Warehouse Logistics

Student: Medvedev Ivan Aleksandrovich

Supervisor: Sanochkin Yuriy Ilyich

Moscow 2025

ABSTRACT AND KEYWORDS

This thesis addresses the challenge of uneven flow of incoming / outgoing goods in warehouse logistics by developing a ML-based system to predict goods flow volumes. Accurate forecasting of goods receipt and shipped enables optimized resource allocation (staff, equipment, space), reducing costs from overstaffing and mitigating operational failures caused by undercapacity.

Using historical data from a medium-to-large non-food retail warehouse, sourced via Warehouse Management System (WMS), the study implements and compares:

1. Statistical models (ARIMA, SARIMA, SARIMAX),
2. ML models (Lasso, Random Forest, Gradient Boosting, CatBoost),
3. Deep learning architectures (LSTM, GRU, Seq2Seq with attention), and
4. Hybrid approaches (e.g., SARIMA + LSTM).

Forecasts are generated for two horizons: hourly (shift scheduling) and daily (logistic optimization).

Key results:

- CatBoost achieved the best performance for daily predictions (RMSE: 0.096–0.109).
- Seq2Seq with attention excelled in hourly shipment forecasts (RMSE: 0.170).
- Hybrid models improved medium-term shipment accuracy by capturing seasonality and nonlinear trends.

The solution is deployed as a REST API via FastAPI, integrating forecasts into warehouse management workflows. Economic analysis estimates potential savings of 5.4 million rubles/year through labor optimization alone.

Keywords: Warehouse logistics, predictive analytics, time series forecasting, machine learning, CatBoost, LSTM, resource optimization, operational efficiency.

TABLE OF CONTENTS

ABSTRACT AND KEYWORDS 2
INTRODUCTION 4
CHAPTER 1. OVERVIEW OF SUBJECT AREA,
BUSINESS GOALS AND CONSIDERED APPROACHES 6
1.1 Subject Area: 6
1.2 Business Goals, Stakeholders, and Economic effect 6
1.3 Existing Solutions 7
1.4 Possible Approaches to Solve the Stated Purpose of the Work 8
CHAPTER 2. PROBLEM STATEMENT: TECHNICAL
AND FUNCTIONAL REQUIREMENTS. SOLUTION STEPS 13
2.1 Technical and Functional Requirements. Solution Stages 13
2.2 Solution Architecture: Methods and Tools 14
2.3 Solution Economics 15
CHAPTER 3. IMPLEMENTATION OF A COMPREHENSIVE ML SYSTEM
FOR PREDICTING INCOMING AND OUTGOING FLOWS OF GOODS
IN A WAREHOUSE 15
3.1 Data Uploading and Preprocessing 15
3.2 Models Training 21
3.3 Solution Integration 31
CONCLUSION 32
Conclusions on the Work Done 32
Proposals of Further Development Directions 34
REFERENCES 35
APPENDIX. LINK TO GITHUB REPOSITORY 37

INTRODUCTION

Relevance of the work. One of the foundations of modern civilization is the logistics of goods. In turn, it is impossible without efficiently operating warehouses.

The efficient operation of warehouses depends to a large extent on the ability to cope with uneven loads. The key processes shaping warehouse workload are receiving incoming goods and shipping outgoing goods. Therefor resources allocated for managing warehouse operations have to be as "just right" as possible, with flexible adaptation to increase and decrease of workload.

Overestimating of workload leads to inefficient resources booking (staff, warehouse equipment such as forklift trucks, space of warehouse racking etc) which obviously leads to unnecessary costs and decrease of efficiency

In turn, underestimating of workload leads to operations failures (such as failure to unload incoming transport or load outgoing transport, and losses may be high or even extreme (in case failure to load or unload leads to spoilage of goods or sales failures).

Uncertainty may lead to both of the mentioned negative business consequences.

Purpose of the current work is to provide a first version of a complex ML-based tool allowing Warehouses to predict incoming and outgoing goods flow. Such prediction should be used by warehouse operational management for:

- Avoid operations failures due to resource shortage during load peaks not predicted timely and
- Reduce costs for booking resources not required in the periods of low workloads.

Business justifications of the objectives. In order to convert predictions to cost saving obviously management actions are required, and for different prediction horizons different actions are appropriate, i.e.:

- On very short-term period (shift) shift leader may concentrate on relocating shift staff either on current workload or on long-term supplementary tasks such as stocktaking, picking area compaction, maintenance etc.
- On short term period (next few shifts) shift leader and warehouse manager is able to increase or decrease the number of temporary staff, which is common practice in warehousing business.
- On medium term period (next few weeks and more) warehouse manager is able to reduce or increase number of total staff and rented equipment available.

Objectives of the work are:

- to explore various ways for short- and medium-term predictions of incoming and outgoing goods flows on a warehouse based on historical data and

- make a working tool using approach that utilizes best approaches.

Prediction objects, methods and technologies. Author of this thesis has NDA access to historical data of incoming and outgoing goods statistics of a medium-to-large non-food retail warehouse that has approx. 150 workers/shift. Data is retrieved from WMS (Warehouse Management System).

The following models to prediction of incoming/outgoing warehouse goods are tested:

- Statistical models (ARIMA, SARIMA, SARIMAX),
- ML models (Lasso, Random Forest, Gradient Boosting, CatBoost),
- Deep learning architectures (LSTM, GRU, Seq2Seq with attention), and
- Hybrid approaches (e.g., SARIMA + LSTM).

The solution is deployed as a REST API via FastAPI, integrating forecasts into corporate workflows.

Outline of the thesis. The thesis consists of three chapters. The first chapter is devoted to a overview of subject area, business goals and possible approaches to solve the stated problem. The second chapter describes technical requirements, solution steps, methods (models) used. The third chapter presents the realization: data preprocessing, training process, prediction results and solution integration.

CHAPTER 1. OVERVIEW OF SUBJECT AREA, BUSINESS GOALS AND CONSIDERED APPROACHES

1.1 Subject Area

The object of forecasting in this paper is the operational load of the warehouse, quantitatively expressed through the volumes of the two main material flows: receiving and shipping. Receiving includes unloading transportation, inspection and registration of incoming goods, their preparation and placement in storage. Shipment covers order picking, packaging, preparation of documents and loading into vehicles. The intensity of these two processes can be measured in various metrics. However most common metrics that can be easily obtained from WMS are:

1) Lines. A line is generally accepted logical unit of processing in WMS. It corresponds to 1 line of a document (e.g. delivery note or shipment note), in which all pieces of goods of one article of one batch (or the same goods without batch accounting) belonging to this document are recorded. An incoming and outgoing document is a group of incoming / outgoing goods combined in one delivery / shipment. Warehouse receiving/shipment processes are carried out on these documents respectively.
2) Goods weight
3) Goods volume

Why ML? Receipt and shipment volumes to-/from a Warehouse are dynamic and stochastic variables shaped by many interrelated factors. Time patterns play a major role: annual and monthly seasonality, weekly and daily cyclicity are present. They are also affected by commercial factors such as marketing promotions, sales, price and assortment changes. Logistical factors also play a role, including supplier delivery schedules and transportation availability, road traffic situations. Therefor various ML tools seem to be appropriate for such predictions.

1.2 Business Goals, Stakeholders, and Economic effect

Business goal of the work is to develop a warehouse load prediction tool that will help:

- Save costs by more accurate resource utilization;
- Save costs and improve quality of warehouse service by avoiding operational failures, i.e. avoid situations as “truck not unloaded in time”.

Stakeholders interested in the results of the work are:

- Senior warehouse management is interested in both positive effects of better prediction cost savings and reduction of operational failures;

- Middle warehouse management is interested in predictions as it can make their daily work easier by decreasing the number of out-of-control situations. Better predictions used for more effective management may as well increase their personal KPIs.

Economic effect

Author was not able to find any reliable estimation of warehouse losses specifically due to overestimation and underestimation of goods flow.

However, basing on operational warehouse experience author made a rough evaluation calculation of possible cost saving due to decrease of warehouse staff overspending:

Cost of one 12-hour shift of logistic worker is about 5K rubles including taxes. On a typical medium-large retail warehouse such as one considered in this work there are 100 line workers per shift, in total 4 shifts for 24/7 operations.

Assuming that overspending is 10% and basing on rather modest assumption that better prediction with decent management actions may lead to decrease of overspending by 30% annual saving may comprise 5.4 mln rubles annually. Possible savings due to decrease of operational failures are hard to estimate without specific warehouse conditions, but such savings would be on top of labor costs saving.

Therefore, even rough estimation proves that economic impact might be significant.

There are also indirect / hard to calculate benefits of using effective prediction system, specifically:

- Decreasing periods of excessive load and “slack time” for staff decreases stress and makes up better labor conditions, which leads to better morale and more effective work;
- As a general rule more predictable work is more efficient, being proactive is more effective than just being reactive.
- Better forecasting improves customer service

In a whole implementation of an effective forecasting system not only provides direct economic effect, but also contributes to strategic business goals.

1.3 Existing Solutions

At the moment I have not found solutions providing a full-fledged ML forecasting module on the Russian market of warehouse management systems. Key players such as 1C:WMS[12], TopLog WMS[13], Axelot WMS X5[23] offer reports and analytics for expert forecasting. Ailogica not being a warehouse management system offers integration with ERP system and with the help of machine learning and AI techniques to improve enterprise operations. “By implementing an AI assistant the company has reduced excess inventory by 32% and reduced

storage and logistics costs by 20%." - assures Ailogica[10]. But this is actually another application of AI in warehouse management.

If we look at the international market, there are warehouse management systems offering a forecasting module. For example, Blue Yonder use AI/ML for demand forecasting, inventory optimization and warehouse operations management, including labor forecasting. Manhattan Associates focus on labor optimization (Labor Management), which is closely related to load forecasting[11]. However author had not found any specific cases of successful implementation of ML-based predictions implemented, although there are many articles about such possibilities, and declarations of such AI applications.

My conclusion is that this topic is quite well-developed, most of WMS manufacturers and Warehouse companies currently only explore/prepare for application of ML, including prediction of flows.

1.4 Possible Approaches to Solve the Stated Purpose of the Work

Since the load on the warehouse, namely the volumes of incoming and outgoing deliveries, is primarily distributed in time, the forecasting of the load on the warehouse is reduced to the forecasting of time series. Below I consider the chronological development of methods for solving this problem.

1.4.1 Traditional Statistical Methods and Corresponding Models:

Moving average methods: The forecast for the next period is calculated as an average of several previous periods. Easy to implement, but do not adapt well to trends and seasonality.

Exponential smoothing: An improvement on moving average, where more recent data is given more weight. These methods perform well for data with clear seasonal and trend components.

ARIMA/SARIMA (Autoregressive Integrated Moving Average / Seasonal ARIMA) models: A classic and powerful tool for analyzing and forecasting time series. The SARIMA model explicitly accounts for seasonal variations, which is often found in warehouse load data (daily, weekly, annual cycles). These models are based on autocorrelation of the data and require assumptions about the stationarity of the series (or making it stationary). They provide good interpretability but can be difficult to set up and do not always deal well with non-linear dependencies or the influence of external factors without explicitly including them (ARIMAX).

These statistical approaches often serve as base models for comparison with more sophisticated methods. They have the advantage of relative simplicity and interpretability.

However, they may not be flexible enough to capture complex patterns, non-linear dependencies, and the influence of multiple external factors.

1.4.2 Machine Learning

Machine learning methods have become increasingly widespread recently with the development of computing power and data availability. Unlike classical statistical models, they often do not require strict assumptions about data structure and can automatically detect complex dependencies. The forecasting task can be viewed as a regression task, where the target variable is the load volume, and the attributes are time characteristics (hour, day of the week, month), time series lags, and external factors. Both linear models and more complex models such as those based on decision trees are used [14,15].

Decision Trees and Ensembles: Models like Random Forest and especially Gradient Boosting Machines (GBM) such as XGBoost, LightGBM, CatBoost[16] show high performance in prediction problems on tabular data. They are able to model nonlinear dependencies, feature interactions and often do not require complex data preprocessing (e.g., scaling). They can be effectively applied for both one-step-ahead forecasting and for predicting a sequence of values.

ML methods allow the integration of a large number of heterogeneous features and often provide higher accuracy than traditional models, especially on data with complex structure. However, they can be less interpretable and require careful selection of hyperparameters.

There are several approaches to time series forecasting:

Recursive strategy. In this approach, the model only predicts the value of the next time step. To forecast the second step, the previously obtained forecast is used as the value of the previous time step in the feature space. Similar actions are performed for the next steps, constantly augmenting the feature space with predictions (see Fig. 1). This strategy allows us to obtain forecasts for any horizon, but the disadvantage of this approach is the accumulation of error with increasing forecast horizon.

Testing Set to forecast M+1

Row Id	M-2	M-1	M	M+1 (Target)
10	74	89	122	XXX

Testing Set to forecast M+2

Row Id	M-2	M-1	M	M+1 (Target)
11	89	122	XXX	YYY

Testing Set to forecast M+3

Row Id	M-2	M-1	M	M+1 (Target)
12	122	XXX	YYY	ZZZ

Figure 1. The feature space under recursive strategy. [14]

Direct strategy. In the direct strategy, it is assumed that each forecast within the forecast horizon should be built by a different model. Thus, for forecasting for a horizon of length N, it is necessary to train N models with the same feature space but with different target variables (see Fig. 2). The forecasts are obtained as follows: the first model makes a forecast for the next step, the second model makes a forecast for the next step, the third model makes a forecast for 2 steps later, and so on. This approach reduces the accumulation of error, but requires more resources for model training.

Training Set (M+1 model)

Row Id	M-2	M-1	M	M+1
1	94	125	62	57
2	125	62	57	92
3	62	57	92	134
4	57	92	134	120
5	92	134	120	134
6	134	120	134	132
7	120	134	132	74
8	134	132	74	89
9	132	74	89	122

Training Set (M+2 model)

Row Id	M-2	M-1	M	M+2
1	94	125	62	92
2	125	62	57	134
3	62	57	92	120
4	57	92	134	134
5	92	134	120	132
6	134	120	134	74
7	120	134	132	89
8	134	132	74	122

Training Set (M+3 model)

Row Id	M-2	M-1	M	M+3
1	94	125	62	134
2	125	62	57	120
3	62	57	92	134
4	57	92	134	132
5	92	134	120	74
6	134	120	134	89
7	120	134	132	122

Testing Set (M+1 model)

Row Id	M-2	M-1	M	M+1
10	74	89	122	?

Testing Set (M+2 model)

Row Id	M-2	M-1	M	M+2
10	74	89	122	?

Testing Set (M+3 model)

Row Id	M-2	M-1	M	M+3
10	74	89	122	?

Figure 2. The feature space under recursive strategy. [14]

Hybrid strategy. Hybrid strategy has a similar algorithm to the direct strategy, it also trains N models for forecasting for a horizon of size N with the difference that each next model uses the forecasts of the previous ones (see Fig. 3).

Training Set (M+1 model)

Row Id	M-2	M-1	M	M+1
1	94	125	62	57
2	125	62	57	92
3	62	57	92	134
4	57	92	134	120
5	92	134	120	134
6	134	120	134	132
7	120	134	132	74
8	134	132	74	89
9	132	74	89	122

Training Set (M+2 model)

Row Id	M-3	M-2	M-1	M	M+1
1	94	125	62	57	92
2	125	62	57	92	134
3	62	57	92	134	120
4	57	92	134	120	134
5	92	134	120	134	132
6	134	120	134	132	74
7	120	134	132	74	89
8	134	132	74	89	122

Training Set (M+3 model)

Row Id	M-4	M-3	M-2	M-1	M	M+1
1	94	125	62	57	92	134
2	125	62	57	92	134	120
3	62	57	92	134	120	134
4	57	92	134	120	134	132
5	92	134	120	134	132	74
6	134	120	134	132	74	89
7	120	134	132	74	89	122

Testing Set (M+1 model)

Row Id	M-2	M-1	M	M+1
10	74	89	122	XXX

Testing Set (M+2 model)

Row Id	M-3	M-2	M-1	M	M+1
9	74	89	122	XXX	YYY

Testing Set (M+3 model)

Row Id	M-4	M-3	M-2	M-1	M	M+1
8	74	89	122	XXX	YYY	ZZZ

Figure 2. The feature space under recursive strategy. [14]

1.4.3 Deep Learning

DL models have the potential to provide the highest accuracy, especially on large amounts of data with complex, nonlinear and multivariate dependencies. However, they require significant computational resources for training and large amounts of data, and are difficult to set up and interpret.

Recurrent Neural Networks (RNN). Models such as LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) are specifically designed to handle sequential data. They are able to capture long-term dependencies in time series, which is important for load forecasting dependent on past events. Their advantage is their ability to train on “raw” time series without complex feature engineering[9].

New architectures such as Transformer[17] were originally designed for natural language processing but are adapted for time series as well. Thanks to an attention mechanism that allows the model to weight the importance of different parts of the series history. Studies show the superiority of Transformers in large-window forecasting due to the attention mechanism. At the same time, GRU and LSTM[9] performed the best when there is a lack of data and in the forecasting task with a small window.

Here are some modern transformers for time series forecasting that are considered best[3]:

- Chronos, an open source family of pre-trained time series forecasting models from Amazon[1]
- TimesFM - an open source model from Google Research, pre-trained on over 100 billion real time series points[2,7]
- Moirai - a model from Salesforce AI Research, trained on 27 billion observations from 9 separate domains[8]
- TimeGPT - a model from Nixtla that takes into account the influence of external variables[4]

1.4.4 Hybrid approaches and ensembles

The current trend is to use hybrid models that combine the strengths of different approaches. For example:

- SARIMA + ML/DL for modeling residuals,
- a combination of DL for extracting features from the sequence and a separate model for the final forecast based on all features,
- ensemble forecasts from several models to improve the accuracy of the final forecast.

Thus, using time series decomposition into components and using a hybrid model based on LSTM+SARIMA allowed to reduce the error by half in the work of Shizhe Li[16].

1.4.5 Modern Experiments

Bi-Mamba+ is a modern model for Long-Term Time Series Forecasting (LTSF) that combines deep learning techniques and state model to handle long dependencies and improve computational efficiency [5].

The input time series is divided into patches, which helps to account for both inter-series and intra-series dependencies in more detail. The data is processed both forward and backward, which improves the capture of relationships in the data. Spearman's correlation coefficient is used to choose between independent and mixed tokenization, depending on the data. The model utilizes a forgetting mechanism to help retain information about the past, improving predictions over long time intervals. Bi-Mamba+ is more efficient compared to transformers, providing less resource consumption when processing long sequences.

Experiments on real datasets (weather, traffic, energy) show that the model outperforms other state-of-the-art approaches in terms of forecast accuracy, including transformer-based models.

TIME-LLM is a technique for adapting large language models for time series prediction through data reprogramming [6]. In this model, the time series is converted into textual prototypes, making it more suitable for processing by language models. To improve the model's ability to analyze the data, additional context is introduced through textual cues, which allows for a better understanding of the task. After the data is processed by the language model, the results are projected back to generate predictions.

TIME-LLM consistently outperformed specialized models on standard benchmarks (ETTh1/2, ETTm1/2, Weather, Electricity, Traffic, ILI). On average, TIME-LLM showed a 1.4% reduction in MSE error compared to PatchTST, a 12% reduction compared to DLinear and GPT4TS, and a 20% reduction compared to TimesNet when using Llama-7B. TIME-LLM performed better in 36 of the 40 test cases. TIME-LLM has demonstrated high performance in

situations where limited training data is available, showing excellent results even when trained on 10% or 5% of the total data [6].

CHAPTER 2. PROBLEM STATEMENT: TECHNICAL AND FUNCTIONAL REQUIREMENTS. SOLUTION STEPS

2.1 Technical and Functional Requirements. Solution Stages

The task of warehouse load forecasting is to create a system capable of predicting the receipt and shipment volumes of goods in the warehouse taking into account time characteristics. Basing on future business use prediction is performed on two time horizons:

- hourly (accurate hourly prediction for shift scheduling)
- daily (total day prediction for overall logistic optimization). For this task, an approach combining machine learning and deep learning will be used to efficiently predict dynamic data based on historical data.

In order to achieve the objective, the following steps should be carried out sequentially:

2.1.1 Data Collection, Processing and Exploratory Analysis (EDA)

- Aggregation of historical data on receiving and shipping operations from the warehouse information systems (WMS).
- Data cleaning: handling omissions, outliers, errors.
- Data conversion into time series with hourly and daily aggregation.
- Feature engineering: creating additional features such as lag variables, moving averages, calendar features (day of the week, month, holidays), etc.
- Conduct exploratory analysis to identify patterns, trends, seasonality, autocorrelation, and relationships between features and target variables (acceptance/shipment volumes).

2.1.2 Model Development, Training and Testing

- Research and implementation of various approaches to time series forecasting, including both classical ML methods (such as ARIMA, linear models, wooden models - RandomForest, GradientBoosting, CatBoost) and DL architectures (e.g., RNN - LSTM, GRU; Encoder-Decoder).
- Developing separate models to address two tasks: hourly forecast and daily totals forecast.
- Conducting experiments: training models on historical data, selection of hyperparameters using cross-validation methods, at different forecasting horizons.

- Comparative evaluation of model performance on a test sample using quality metrics (MAE, RMSE).

2.1.3 Selection of the Best Model, System Development and Deployment:

- Selection and registration of optimal models based on metrics of quality, interpretability, and computational cost for each of the forecasting tasks (hourly and daily).
- Development of the server side of the application (backend) encapsulating the selected models and forecasting logic, providing API for access to forecasts.
- Development of the client part - web-interface for user interaction with the system: input of parameters (e.g. forecasting period), visualization of historical data and obtained forecasts.
- Backend and frontend integration, testing the performance of the whole system.

2.2 Solution Architecture: Methods and Tools

The architecture of the developed solution for warehouse load forecasting includes several interrelated components that realize the stages from data processing to providing forecasts to the user. The architecture is based on the application of modern data analysis and machine learning methods implemented using specialized libraries and frameworks..

- Programming Language: Python is the primary development language due to its extensive ecosystem of libraries for data analysis and machine learning.
- Data collection and storage: PostgreSQL, sqlAlchemy.
- Data processing and computation: Pandas (data structures and analysis tools), NumPy (numerical computing).
- Visualization: Matplotlib, Seaborn - graphing for EDA and presentation of results.
- ML: Scikit-learn (data preprocessing, metrics, implementation of some models), Statsmodels (ARIMA/SARIMA), CatBoost (implementation of gradient boosting).
- DL: PyTorch - framework for building and training neural networks.
- Backend: FastAPI - creation of API for access to models.
- Containerization: Docker - packaging the application and its dependencies.
- FrontEnd: Can be implemented using modern JavaScript frameworks (e.g. React, Vue, Angular).
- Solution lifecycle management: git - versioning of application source code. MLflow - model registration

2.3 Solution Economics

The implementation of a warehouse load forecasting system requires consideration of both direct and indirect costs, which include the development, implementation and operation of the system. As common practice such costs are aggregated in hourly rate of IT development company or internal IT department.

See a rough estimation of possible costs calculation in table 1.

Table 1. Sample costs calculation for implementation of the solution.

Cost type	Cost	hours	hourly rate, K rubl	total, K rubl
Development costs				
	Initial development	210	3,1	651
	Integration with WMS system	20	3,1	62
Implementation costs				
	On site training	16	3,1	50
	Trial operations start & support	32	3,1	99
	Improvements based on trial operations feedback from business users	80	3,1	248
Support costs				
	Technical support for 12 months	40	3,1	124
TOTAL COSTS				**1 234**

As the system has to be validated in fulfilling business goals for specific warehouse, this calculation cannot be considered as valid for any or majority of the cases, it's done for evaluation purposes only.

However, considering that economic payback according to rough estimation may exceed 5 mln rubl / year for labor cost saving only (see paragraph 1.2.) economics of the solution is quite promising.

CHAPTER 3. IMPLEMENTATION OF A COMPREHENSIVE ML SYSTEM FOR PREDICTING INCOMING AND OUTGOING FLOWS OF GOODS IN A WAREHOUSE

3.1 Data Uploading and Preprocessing

In the process of model development, data from the warehouse management system (WMS) integrated with the company's ERP system were used to forecast the warehouse load. Each

operation of goods receipt is accompanied by an ASN (Advanced Shipping Notice) document. Goods shipment is accompanied with Shipment Order (SO). These two documents are the main source of data. Information about each ASN/SO document is transferred from the ERP to the WMS, where it is stored in a structured form for further analysis. The ASN/SO documents, being the basic element for each receipt or shipment, include information about the items, their volumes and weights. Each line of the document corresponds to an individual item type (or type & batch if applicable) that is received into or shipped out of the warehouse.

Receipt and shipment data were organized into two separate tables, one containing information on receipts (incoming goods flow) and the other on shipments (outgoing goods flow).

Author of this thesis paper has NDA access to such data from medium-large non-food retail warehouse (further in the paper "Warehouse").

The tables contain the following key fields:

- date - date of the transaction (acceptance or shipment);
- time - time of the operation;
- incoming/outgoing lines - number of lines in the ASN document, i.e. number of goods items in the corresponding operation (acceptance or shipment);
- incoming/outgoing volume - volume of goods specified in the ASN document for the corresponding operation;
- incoming/outgoing brutto - gross weight of goods specified in the ASN document for the corresponding operation.

It is important to note that all operations with goods are recorded in real time, which allows you to get up-to-date data on each receipt or shipment. The data is received by the WMS with accuracy to the minute, which makes it possible to forecast the load on the warehouse in detail both by time intervals (by hours) and by total data for a day.

After uploading data on receipts and shipments of the Warehouse in the period from 01.05.2023 to 04.12.2024 two DataFrames were obtained. There were totally 126 377 receipts and 115 526 shipments during this period.

Statistical analysis of key metrics showed significant variations in the data, which is an important factor for predicting the warehouse workload. For example, the average value for the number of receipt lines is 60.19, but the maximum values reach 15 411 lines, indicating infrequent large shipments entering the warehouse. A similar situation is observed in the shipment data where the maximum number of rows reaches 1 479.

The volumes of goods entering the warehouse range from 0 to 5 thousand m^3, while the shipments range from 0 to 2.68 thousand m^3. These variations in the data can be due to seasonal

fluctuations in demand, internal warehouse processes, or external factors affecting the intensity of operations. The standard deviation for most metrics indicates a high variability in operations.

Таблица 2. Основные статистики исходных данных

	Incoming lines	**Incoming volume**	**Incoming brutto**	**Outgoing lines**	**Outgoing volume**	**Outgoing brutto**
mean	60.1939	6.686397e+06	885.7380	107.150279	7.225853e+06	955.1588
std	109.7389	2.216082e+07	2266.3216	116.761336	1.373886e+07	1552.1196
min	1	0	0	1	0	0
25%	7	1.2709e+05	18.59	28	1.047629e+06	133.297
50%	23	9.0648e+05	116.517	70	3.400130e+06	436.665
75%	72	5.377785e+06	654.154	137	8.459459e+06	1164.365
max	15 411	5.204237e+09	183296.452	1479	2.684273e+09	44536.563

Basic units in WMS are: cm^3 for volume and kg for weight.

For ease of analysis and further use of the data, cm3 were converted to m3 and kg to tons, volume values were divided by 1e+06 and the weight by 1e+03. This transformation reduced the scale of the numbers, improving readability and simplifying data processing, especially for aggregation and model building. Following this transformation, the data were aggregated by both day and hour, yielding two types of tables for receipts and shipments. When aggregating the data, we get additional statistics in the form of the number of receipts and shipments per time interval (count).

The next step was to eliminate outliers using the z-criterion. Abnormal values for all statistics were replaced by the average value. It is also worth mentioning the omissions that appeared after resampling by days and hours. For example, in the data aggregated by day, there are no receipts and shipments only on January 01, which is due to the celebration of the new year and a day off. For hourly aggregation 2351 hours for receipts 1720 hours for shipments were missing, which indicates that the processing of goods during the day is not constant.

The main metric for us is volume, as it's natural for warehouse storage and processing of goods. The figure below (Figure 4) shows that volume is highly correlated with weight, especially in shipments, which may be due to the need for efficient use of space in machines.

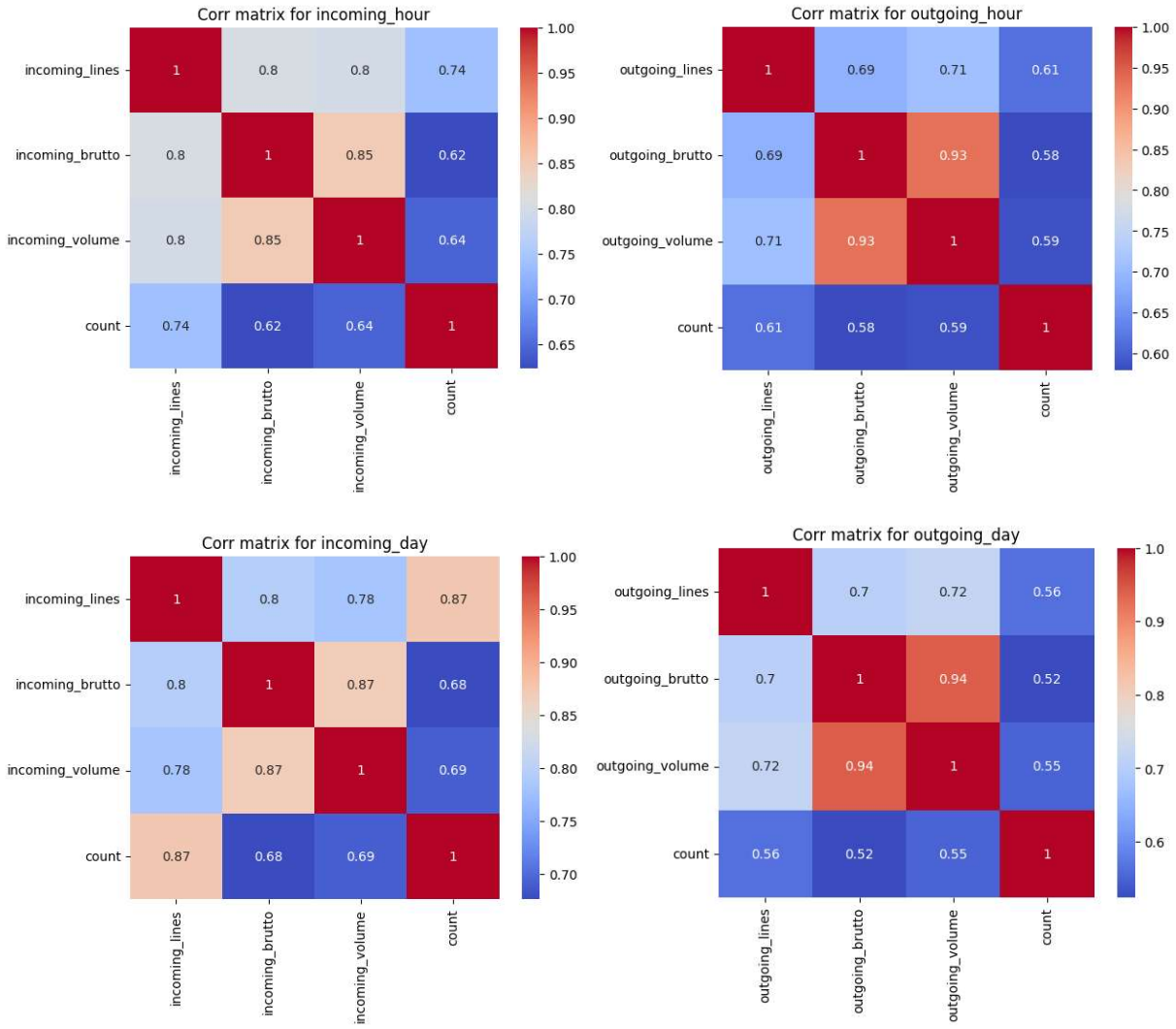

Figure 4. Corellation matrices

For further analysis it is necessary to check the presence of stationarity of the analyzed series. The Dickey-Fuller test showed that all the analyzed series are stationary (p_value<0.05), which allows us to conduct tests providing for stationarity without using differentiation of the series.

For the analysis of acceptance volume data, the ACF and PACF visualization showed strong cyclicality and dependence at the hour level and weak dependence at the day level. The graph of ACF by hour shows clear seasonality with pronounced peaks, indicating regular fluctuations in the acceptance data during the day. These fluctuations may be associated with specific hours of high or low activity in the warehouse. The graph of PACF by hour (see Figure 5) confirms the presence of short-term dependencies, excluding the influence of intermediate values, indicating a high correlation between neighboring hours. In turn, the plots for ACF and PACF by day show weak dependence at the day level, where correlations decrease rapidly, suggesting that there is no pronounced seasonality or long-term trends in the acceptance volume data at the day

level. This may indicate less stability and more irregular changes in the flow of goods in the warehouse in the long run.

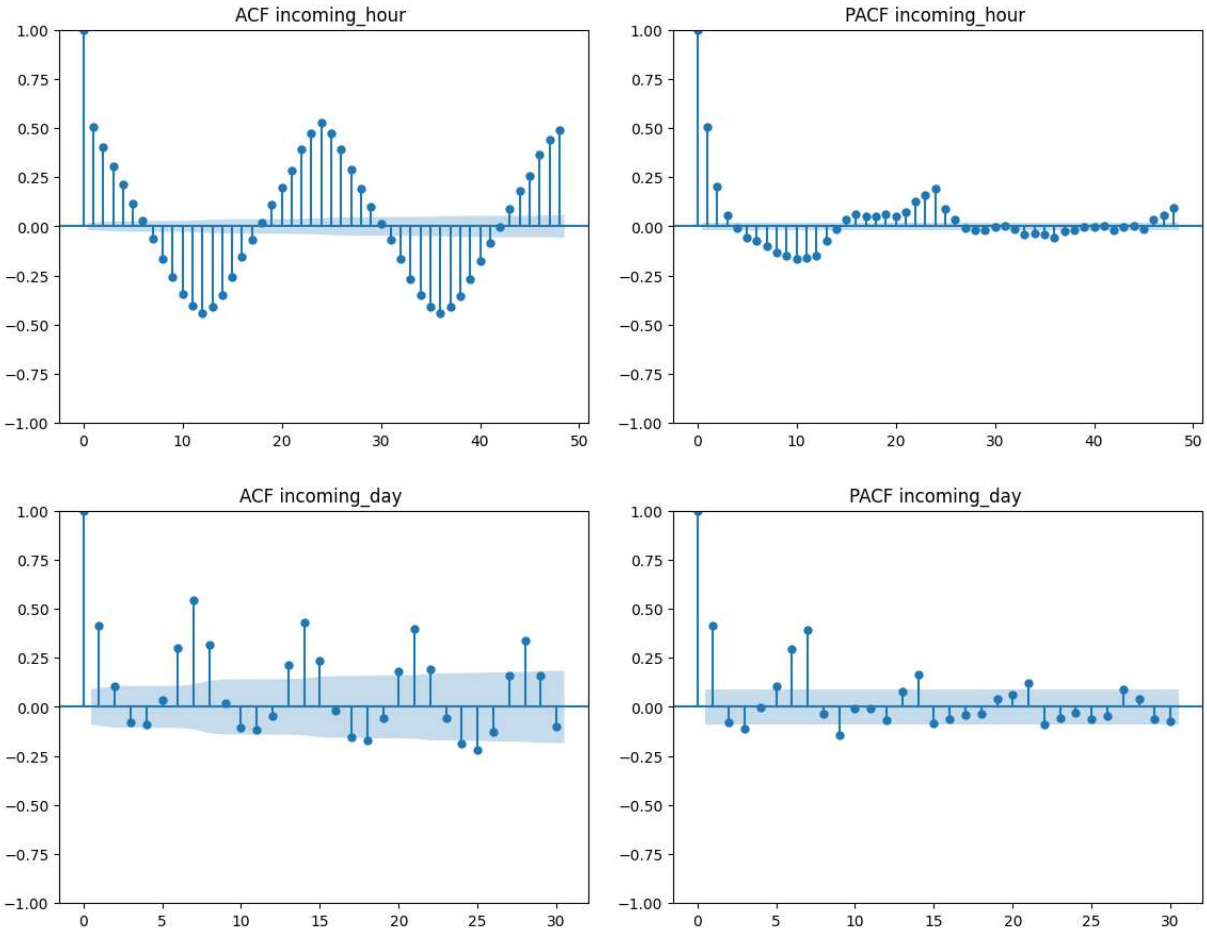

Figure 5. ACF and PACF for receipt volume time serie.

The ACF visualization for shipments by hours (see Fig. 6) showed a clear cyclical pattern with pronounced peaks, indicating the presence of seasonal fluctuations and a strong dependence between current and previous values at the hour level. The PACF graph (see Figure 6) confirmed the short-term dependencies between shipment values, excluding the influence of intermediate lags. In contrast to the hourly data, the ACF for shipments by day showed weak autocorrelation, indicating low long-term dependence and less seasonality in the behavior of shipments at the day level. The PACF for day data also showed a rapid decline in influence after the first lags, indicating limited short-term dependence.

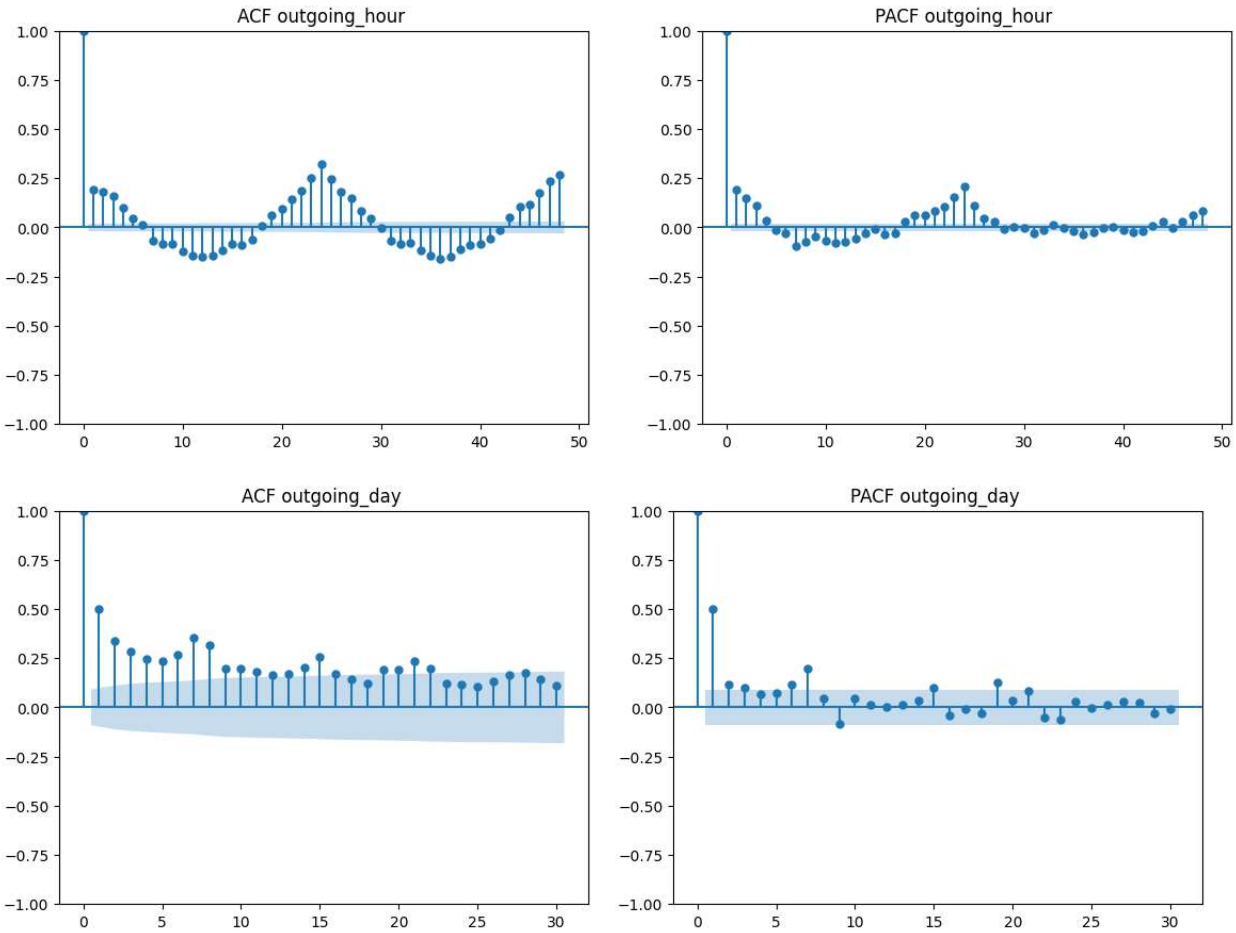

Figure 6. ACF and PACF for shipments volume time series

Looking at the dependence of the average volumes of goods accepted and shipped in the warehouse (see Fig. 7-8), one can notice certain patterns of warehouse operation. For example, the main volume of receipts acceptances falls on the day shift, while the majority of shipments were completely overnight. This is also indicated by the distribution of missing values. The predominance of volumes on weekdays is characteristic of the two processes, although it is less pronounced for shipments.

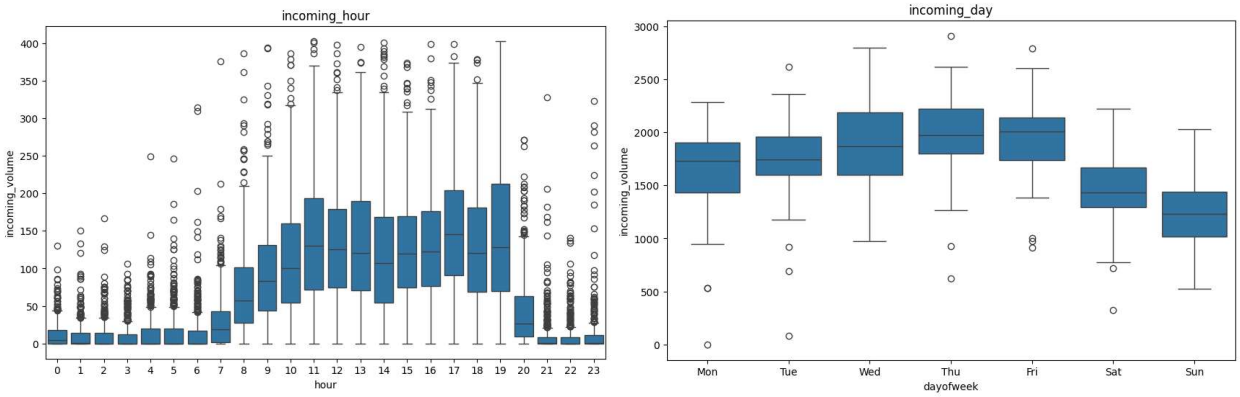

Figure 7. Distribution of receipt volumes by time and for the week.

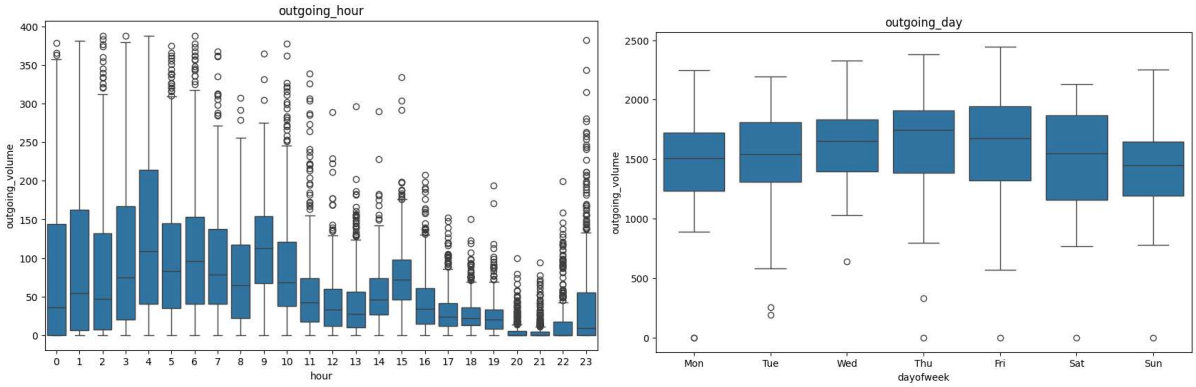

Figure 8. Distribution of receipt volumes by time and for the week

As a result of this work, a comprehensive approach to the preparation and analysis of data for forecasting warehouse load based on information from integrated WMS and ERP systems was implemented. Statistical analysis revealed significant variability of key indicators due to both internal features of warehouse processes and external factors, and data transformation and aggregation helped to simplify subsequent processing. Outliers were identified and eliminated, outliers were treated, and correlation analysis was performed, showing a high correlation between volume and weight of goods. Checking the stationarity of time series and analyzing the autocorrelation functions allowed us to identify pronounced cyclicality and short-term dependencies, as well as to determine the characteristic patterns of warehouse operation, which confirms the correctness of the chosen approach and creates the basis for building an effective predictive model at the next stages of the study.

3.2 Models Training

3.2.1 Statistical Models

All the necessary functionality for working with ARMA family models is provided by the python library statsmodel[19]. The conducted Dickey-Fuller test confirmed the stationarity of all investigated series (p-value < 0.05), which allows using ARIMA models without prior differentiation. In this paragraph I consider the following models:

- *ARIMA.* The basic ARIMA(p,d,q) model was implemented with the parameter d=0, since the series are already stationary. Ranges were used to select optimal values of p and q, which is justified by the dependencies revealed in the correlation analysis.
- *SARIMA.* An extension of the basic ARIMA was the SARIMA model, which takes into account seasonal components in the data, which is an important component of the analyzed series. The seasonal component SARIMA(p,d,q)(P,D,Q)_s with periods s=7

(for daily data) and s=24 (for hourly data) should help the model to correctly describe the cycles of warehouse activity.

- *SARIMAX.* This model is an extension of the SARIMA model, providing an opportunity to take into account external factors. The time characteristics chosen as external factors are: day of the month, quarter, day of the week, month and hour (for hourly data). It is also possible to add additional data to the exogenous variables, such as data on orders and promotions. The main limitation of using external factors is that they are used not only in training but also in forecasting, which makes it impossible to use data from another business process.

The selection of optimal parameters of each model is realized through the enumeration of different combinations of p, d and q values in the given ranges for each sampling level. The Akaike's criterion (AIC) is used as an estimate of model quality, which provides a balance between approximation accuracy and model complexity.

$$AIC = 2k - 2ln(L)$$

where k is the number of model parameters and L is the maximum value of the model likelihood function.

Eighty percent of the data without mixing is allocated for training in order to keep the data consistent.

Table 3. Mean value of errors for predicting receipts.

Model	Day resampling		Hour resampling	
	MAE	**RMSE**	**MAE**	**RMSE**
ARIMA	0.182391	0.215612	0.173327	0.257459
SARIMA	0.112741	0.140815	**0.103510**	**0.158161**
SARIMAX	**0.112246**	**0.140362**	**0.103510**	**0.158161**

Table 4. Mean value of errors for predicting shipments.

Model	Day resampling		Hour resampling	
	MAE	**RMSE**	**MAE**	**RMSE**
ARIMA	0.094876	0.123999	0.126420	0.186434
SARIMA	**0.077267**	**0.096190**	**0.118784**	**0.173648**
SARIMAX	0.079211	0.098670	**0.118784**	**0.173648**

The results show (Tables 2-3) that taking into account recurring patterns over time significantly improves the quality of forecasts. The simple ARIMA model performs the worst, while adding seasonality to SARIMA gives a noticeable improvement - errors are reduced by 20-

40% for different types of operations. The SARIMAX model with additional factors (day of the week, hour, etc.) is almost the same as SARIMA, indicating that the main patterns are already accounted for through seasonality. Interestingly, hourly scheduling works better for goods receipts (error 0.104 vs. 0.112), while daily scheduling works better for shipments (0.077 vs. 0.119). This may be due to the fact that receipts are highly dependent on the time of day, while shipments are more evenly distributed throughout the day.

3.2.2 Machine Learning Models

The main difference between the approach to training machine learning models and statistical models is data preprocessing. If for training of ARIMA family models the investigated series was simply used, then for ML models these data should be brought into a "tabular" form. All the necessary functionality for working with models is provided in the Scikit-learn[20] library in python.

Feature preprocessing. Lag features and moving averages of the target variable are used to convey information about past values of the series. The number of blends and window sizes were chosen considering the size of data aggregation (by day and by hour). Since the data revealed certain cyclicities related to the specifics of the company's operations, temporal attributes were added to the feature space. The created variables included year, month, day, day of the week, quarter, as well as hour for hourly data and a binary indicator of weekends.

Training technology. For convenient training of models and selection of hyperparameters the SklearnTrainer class was implemented, which performs all necessary pipelining - from preprocessing of features to estimation of model quality on a test sample (20%). The implemented class is a universal solution for any ML models, requiring only passing the ranges of hyperparameters for the trained model. The procedure of selecting the most informative features is performed using the Recursive Feature Elimination algorithm, which limits the final feature space to the 15 most significant variables. This strategy is aimed at preventing model overfitting and reducing the computational complexity of forecasting, which has become very critical in hourly load forecasting. Feature pre-selection reduced the hyperparameter fitting time by a factor of approximately 5 without loss of accuracy. The hyperparameter tuning of the models was implemented using the Optuna library, which provides convenient and automated parameter selection. The target function of optimization is the root mean square error (RMSE), calculated within time cross-validation using TimeSeriesSplit specifically for time series, ensuring the informativeness of the metric under conditions of new data augmentation.

Models used. The study examines four classes of machine learning algorithms that demonstrate different approaches to modeling nonlinear dependencies in time series.

- *Lasso-regression* presents a linear baseline approach with L1-regularization acting as baseline.
- *Random Forest* implements an ensemble method based on multiple decision trees, which can efficiently model complex nonlinear interactions between variables without explicit assumptions about the functional form of dependencies.
- *Gradient Boosting* presents an alternative ensemble strategy based on sequential learning, where each successive tree corrects the errors of the previous ones.
- *CatBoost*, developed by Yandex[16], is a modern implementation of Gradient Boosting with optimized processing of categorical features and built-in mechanisms to prevent overtraining.

Quality assessment. The generation of forecasts is carried out by means of a recursive strategy in which the predicted values are sequentially included in the feature space to generate forecasts for subsequent time horizons. This approach ensures the creation of multi-step forecasts while preserving the temporal structure of dependencies, which is critical for the practical application of the models. The forecasting quality is assessed at two time horizons: 30 periods for daily data and 168 periods (24×7) for hourly time series.

Table 5. Mean value of errors for predicting receipts.

Model	Day resampling		Hour resampling	
	MAE	**RMSE**	**MAE**	**RMSE**
Lasso	0.110771	0.138412	0.155886	0.201869
RandomForest	0.086062	0.109702	0.106737	0.155276
GradientBoosting	0.093351	0.120282	0.102026	0.150868
CatBoost	**0.085547**	**0.109277**	**0.100754**	**0.150442**

Table 6. Mean value of errors for predicting shipments.

Model	Day resampling		Hour resampling	
	MAE	**RMSE**	**MAE**	**RMSE**
Lasso	0.095613	0.116025	0.127367	0.179741
RandomForest	0.083377	0.106591	0.119024	0.169815
GradientBoosting	0.080777	0.103776	0.119780	0.170811
CatBoost	**0.074998**	**0.095944**	**0.116333**	**0.168324**

Analysis of the mean values of the accuracy metrics (Tables 4-5) shows the clear superiority of the CatBoost model, which shows the best performance in all prediction scenarios: for acceptances at the day level (RMSE = 0.109), hourly level (RMSE = 0.150), and for shipments

at the day (RMSE = 0.096) and hourly (RMSE = 0.168) levels. Lasso regression shows the worst performance in all scenarios, especially for hourly acceptance prediction (RMSE = 0.202), confirming the low performance of linear methods when dealing with a large number of categorical attributes. Analyzing the dependence of the error on the forecasting step (Fig.), a general pattern is revealed - the error growth with the increase of the forecasting horizon. For each type of operations CatBoost shows the best stability at a long horizon, yielding to RandomForest at short-term forecasting of acceptances (up to 24 hours, Fig. 9-10). It is worth noting that the most important feature for CatBoost was the time data. The model caught the seasonality of the data, producing forecasts weakly dependent on the previous values.

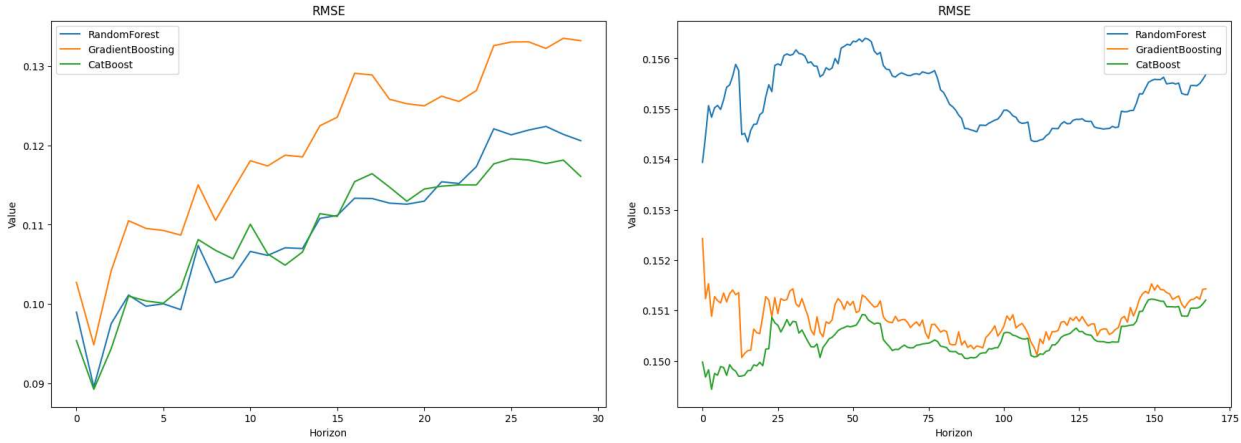

Figure 9. RMSE on predicting receipts by day and by hour.

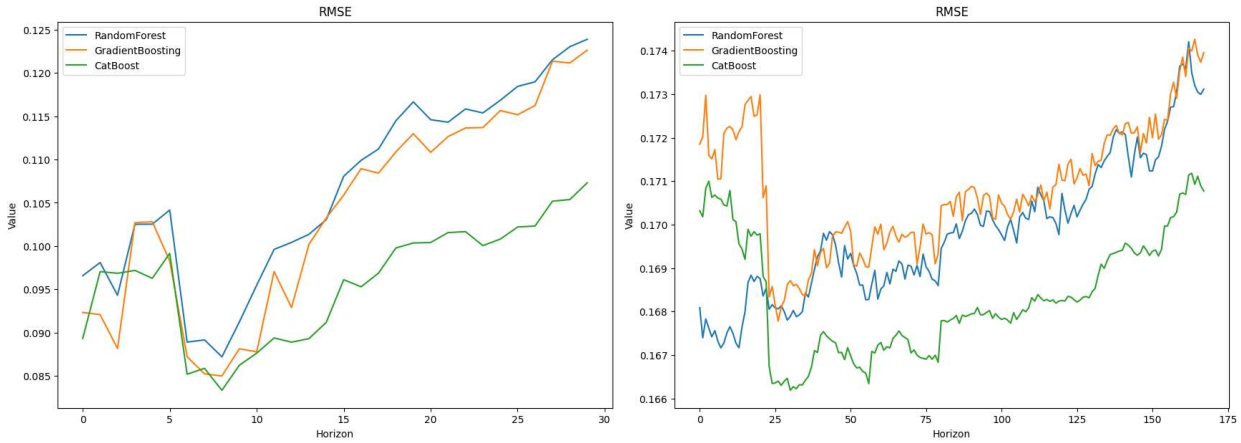

Figure 9. RMSE on predicting shipments by day and by hour.

3.2.3 Deep Learning Models

A fundamental difference between the approach based on recurrent neural networks and classical ML models is the ability to work with data sequences in their natural representation. While traditional algorithms require conversion of time series into a tabular format with lagging features, RNN architectures process sequences directly, preserving the temporal structure of

information through internal memory. All the necessary functionality for model building and training is provided by the PyTorch[21] library in python.

Data preprocessing. For data processing, the Dataset class was implemented to convert the raw data into tensors. Depending on the model used, this class creates tensors of the required length, where each element of the sequence describes the configuration of the warehouse at the moment. Warehouse configuration here refers to receiving and shipping data, as well as attributes describing the temporal component such as month, day, and time (similar to ML processing). One sample is information about already known data (input sequence) as well as features corresponding to future time steps for prediction. Such attributes are data describing the time component, since such data are known in advance without the need for prediction. Also as such attributes can be information about orders, weather - the main limitation, as in the case of exogenous variables in statistical models, such data should be known for the forecast horizon.

Training Technology. All models were built in such a way as to use a single dataset format without the need for refinements. Otherwise, the training format in the case of time series forecasting corresponds to the training of models for working with sequences (e.g., machine translation). All models use a teacher forcing strategy with a customizable coefficient that provides a balance between training stability and autonomous prediction ability. The teacher forcing strategy is based on passing the true sequence values when generating the next step of the sequence, instead of the previously generated ones. A sheduler was also used to gradually decrease the learning rate at each epoch to more accurately adjust the weights at the end of training. All the models described below were trained with the same parameters: input sequence length 14 and 48, prediction horizon length 7 and 48 for day and hour aggregation, respectively.

Architectures used. Four classes of RNN models are implemented in the study, demonstrating different approaches to temporal sequence processing:

- Basic LSTM/GRU models represent fundamental architectures with long short-term memory cells. In this case, the input sequence and the predicted sequence must have the same dimensionality of one element.
- Sequence-to-Sequence architecture implements separate encoder-decoder components to process historical data and generate predictions. The encoder on the GRU generates a comprehensive representation of the dynamics, while the decoder provides step-by-step generation with integration of temporal features. This architecture is chosen in order to give the model more information about the forecasted series. Thus, several time series are fed to the model input (to the encoder). SEQ2SEQ accepts mass and quantity of goods in addition to volume, and

SEQ2SEQ_full accepts data from another business process (both shipping and receiving).

- The attention mechanism model extends the seq2seq architecture with an attention mechanism. The decoder at each prediction step obtains weighted information from the hidden states of the encoder (input sequence), which can be effective for capturing dependencies in warehouse operations. This architecture, similar to seq2seq, is considered in two configurations: with information about one process and about two processes at once.

Quality Assessment. The evaluation approach is consistent with the ML solution. Prediction quality assessment is performed on two time horizons: 30 periods for daily data and 168 periods (24×7) for hourly time series.

Table 7. Mean value of errors for predicting receipts.

Model	Day resampling		Hour resampling	
	MAE	**RMSE**	**MAE**	**RMSE**
GRU	0.091107	0.119682	0.102147	0.151891
LSTM	**0.083404**	**0.109984**	0.102496	0.152856
SEQ2SEQ	0.087689	0.116178	0.102906	0.151439
SEQ2SEQ_full	0.089099	0.117511	0.101801	0.151580
SEQ2SEQ_attention	0.087484	0.113954	**0.101008**	0.152333
SEQ2SEQ_attention_full	0.094194	0.118602	0.101214	**0.151323**

Table 8. Mean value of errors for predicting shipments.

Model	Day resampling		Hour resampling	
	MAE	**RMSE**	**MAE**	**RMSE**
GRU	0.077460	0.094471	**0.117570**	0.172927
LSTM	0.069745	0.087389	0.118759	0.172138
SEQ2SEQ	0.074417	0.095367	0.118211	0.172463
SEQ2SEQ_full	0.070627	0.089888	0.118015	0.173710
SEQ2SEQ_attention	**0.067831**	**0.087101**	0.119962	0.170504
SEQ2SEQ_attention_full	0.097176	0.120479	0.118158	**0.170473**

Analyzing results on tables 6-7 it is noticeable certain advantage in hour forecasting of shipments over ML models. The result worse than CatBoost showed only the model with attention mechanism with full information in the input sequence, which may be due to the lack of training

data and complexity of the model. However, this model demonstrated the best RMSE values (0.15 and 0.17) in hourly forecasting tasks. It is worth noting that models with information about the related business process show the lowest error in forecasting for an interval of up to 24 hours (see Fig. 11-12), showing a better result than CatBoost, but losing to it in medium-term forecasting (0.16 vs. 0.17 in shipments). Continuing the analysis of results it is necessary to note, that at visual estimation of forecasts the regularity is revealed - the models which received only information about predicted process, give more smoothed forecast, in contrast to “full” version which have caught local extremums during day.

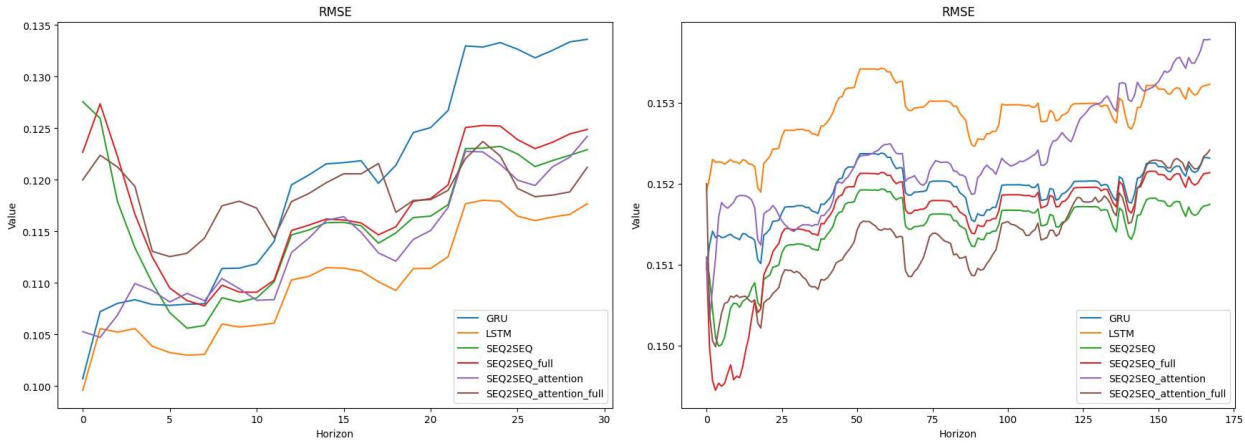

Figure 11. RMSE on predicting receipts by day and by hour.

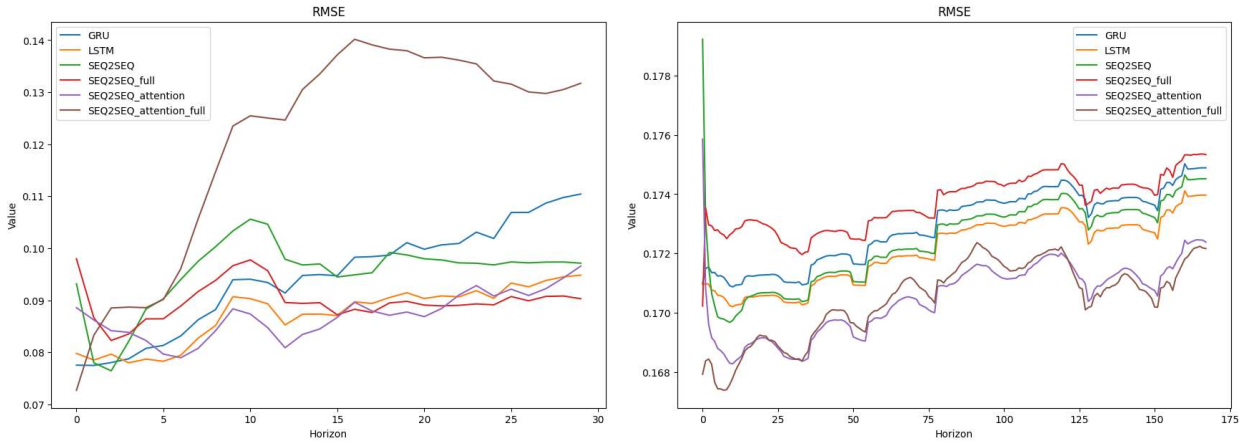

Figure 12. RMSE on predicting shipments by day and by hour.

3.2.4 Hybrid Models

This section discusses a hybrid approach that utilizes previous developments. The main idea is to take a model that captures seasonal phenomena in the time series well, in this case SARIMA, and correct its forecasts with another model that can reveal non-linear dependencies of

the forecasted series. As the second model we will test the recurrent neural networks realized in the previous paragraph, namely:

- GRU
- LSTM
- SEQ2SEQ
- SEQ2SEQ_full
- SEQ2SEQ_attention
- - SEQ2SEQ_attention_full

Data preprocessing. This strategy uses the same approach as the training of recurrent networks described in the previous paragraph. The only difference here is the addition of SARIMA model predictions as exogenous parameters.

Learning technology. For each business process, the previously trained SARIMA statistical models were taken to obtain an initial prediction. The next step is to calculate the residuals from the prediction, which should be further corrected by the trained model. The difference from the precess described earlier is the target variable, here it is the difference between the true volume value and the SARIMA prediction

Quality Assessment. The evaluation approach corresponds to the ML solution. The forecast quality assessment is performed on two time horizons: 30 periods for daily data and 168 periods (24×7) for hourly time series.

Table 9. Mean value of errors for predicting receipts

Model	Day resampling		Hour resampling	
	MAE	**RMSE**	**MAE**	**RMSE**
GRU	0.119936	0.151061	**0.101113**	0.154848
LSTM	0.119936	0.151061	0.101966	0.156829
SEQ2SEQ	0.121097	0.154145	0.102772	0.153615
SEQ2SEQ_full	0.120264	0.150369	0.102331	**0.153565**
SEQ2SEQ_attention	0.120925	0.151513	0.101958	0.154317
SEQ2SEQ_attention_full	**0.114167**	**0.144737**	0.103642	0.153581

Table 10. Mean value of errors for predicting receipts

Model	Day resampling		Hour resampling	
	MAE	**RMSE**	**MAE**	**RMSE**
GRU	0.067865	**0.084424**	0.116206	0.169645
LSTM	0.068194	0.085054	0.115545	0.168348

SEQ2SEQ	**0.067179**	0.086166	0.116310	**0.168275**
SEQ2SEQ_full	0.068054	0.084857	0.115461	0.170483
SEQ2SEQ_attention	0.069511	0.085326	**0.114843**	0.168733
SEQ2SEQ_attention_full	0.070747	0.087847	0.115469	0.169494

The analysis of the results of hybrid models (Tables 8-9) showed that the combination of the SARIMA statistical approach, which well captures seasonal components of the time series, with the capabilities of recurrent neural networks to detect nonlinear dependencies allows to achieve comparable or slightly better quality of forecasting compared to the use of deep learning models alone. In particular, for the task of predicting arrivals at the hour horizon, the hybrid models showed an RMSE of 0.15-0.16, which is close to the best results of the individual RNN architectures and CatBoost. At the same time, on the daytime aggregation, the hybrid models showed the worst result, which is expected, because SARIMA was not able to compete with other models in this process. The greatest quality gain was in the forecasting of shipments, both on the daily and hourly horizon, which makes the hybrid models the best in forecasting series with low autocorrelation, especially in medium-term forecasting (Figures 13-14).

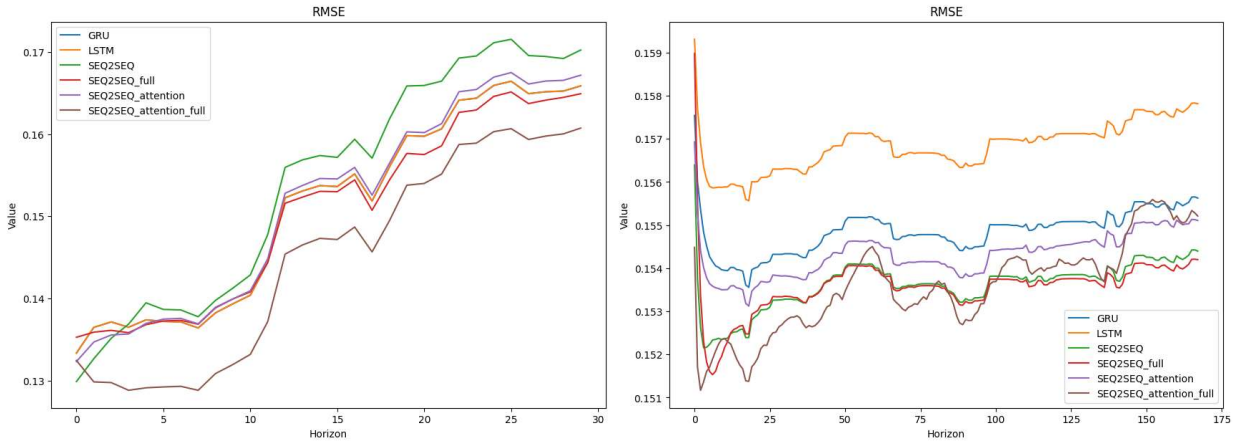

Figure 13: RMSE on forecasting receipts volume by day and by hour.

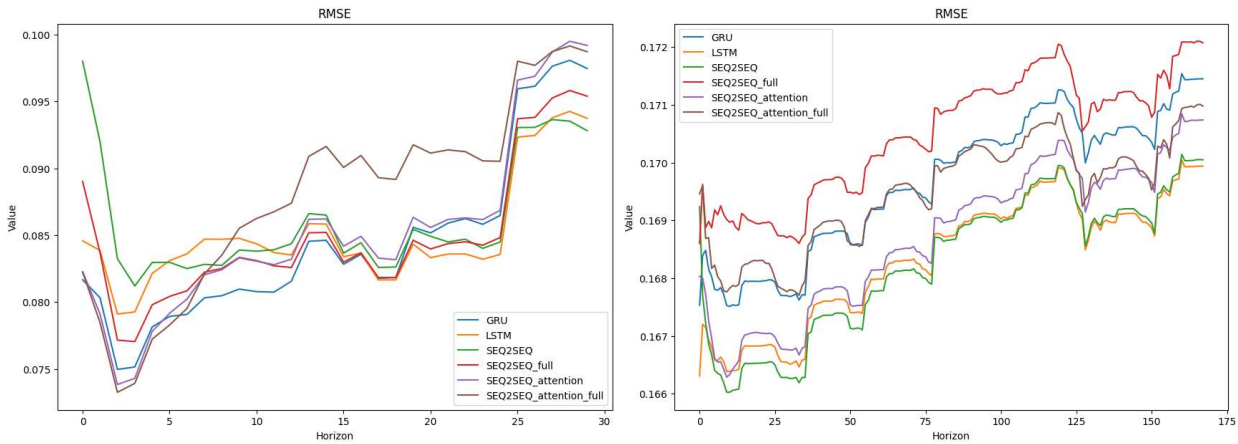

Figure 14: RMSE on forecasting shipments volume by day and by hour.

3.2.5 Conclusion on the Results of Model Comparison

The comparative analysis of the received models shows efficiency and easy selection of hyperparameters of methods of machine learning, in particular CatBoost which has shown the superiority over statistical models of ARMA family and became a strong competitor of recurrent neural networks having shown the smallest value of errors (0.1 MAE and 0.15 RMSE) in the task of hourly forecasting of acceptances. Visual comparison of the forecasts and analysis of feature importance for CatBoost showed that in hourly forecasting, calendar features, especially the categorical feature of the current hour, have the greatest influence, making the forecasts almost independent of past values. Although forecasts of recurrent networks lack such a pattern, and a local extremum appears in the case of adding data from a related business process, their results are comparable to CatBoost errors, which may indicate a small training sample size. The plateau in model training is also indicated by the distribution of errors depending on the horizon, where in the results of all models there is a certain pattern of error increase different only by the offset. It can also be said, given the low autocorrelation of hourly aggregated series, that there are some external factors affecting the workload of the warehouse during the day, such as the number of storekeepers or active orders. In the presence of such data encoder-decoder architectures can show the best results. All the above-mentioned allows to make a choice that the task of forecasting the load of the investigated warehouse is quite solvable and takes place.

3.3 Solution Integration

As part of the development of a warehouse load forecasting system, there is a need to create a server part that would combine all trained models and provide a convenient way to obtain forecasts. To solve this problem in this paper, FastAPI framework was chosen, which allows to quickly create a web service with minimal time and provides high performance. FastAPI is a modern high-performance web framework for Python, specifically designed to create RESTful APIs using standard Python type annotations. A key feature of FastAPI is the auto-generated interactive API documentation using the OpenAPI standard and Swagger UI, which simplifies the integration process with client applications. [22]

FastAPI server acts as an intermediary between trained prediction models and end users of the system. The basic idea is that all models - statistical, machine learning, deep learning, and hybrid - are packaged into a single service that can accept queries and return prediction results in a standardized format.

This approach has several important advantages. First, it provides centralized model management: all algorithms are in one place, which simplifies their updating and maintenance.

Second, it creates a universal interface for obtaining forecasts, regardless of which model is used internally. This means that a web interface or other client applications can work with any of the models through the same queries.

The server is built on the principle of REST API, which means that standard HTTP requests are used to interact with clients. When the application is launched, all necessary models are loaded into the server memory. This includes loading weights of neural networks and machine learning models and parameters of statistical models.

Using FastAPI to create the server side of the prediction system provides high performance and scalability. The framework supports asynchronous processing of requests, which allows to efficiently serve multiple simultaneous connections. This is especially important in an active warehouse environment, when the system must quickly process requests from different users.

The standardized interface simplifies integration with corporate information systems and allows the forecasting results to be used in various warehouse business processes. This makes the developed system not an isolated tool, but a full-fledged component of the enterprise IT infrastructure.

CONCLUSION

Conclusions on the Work Done

Basing on achieved results the following conclusions could be made:

1) Good quality of prediction was achieved for goods flow for 12-hour shifts and longer time intervals (days) (See Figures 15-16):

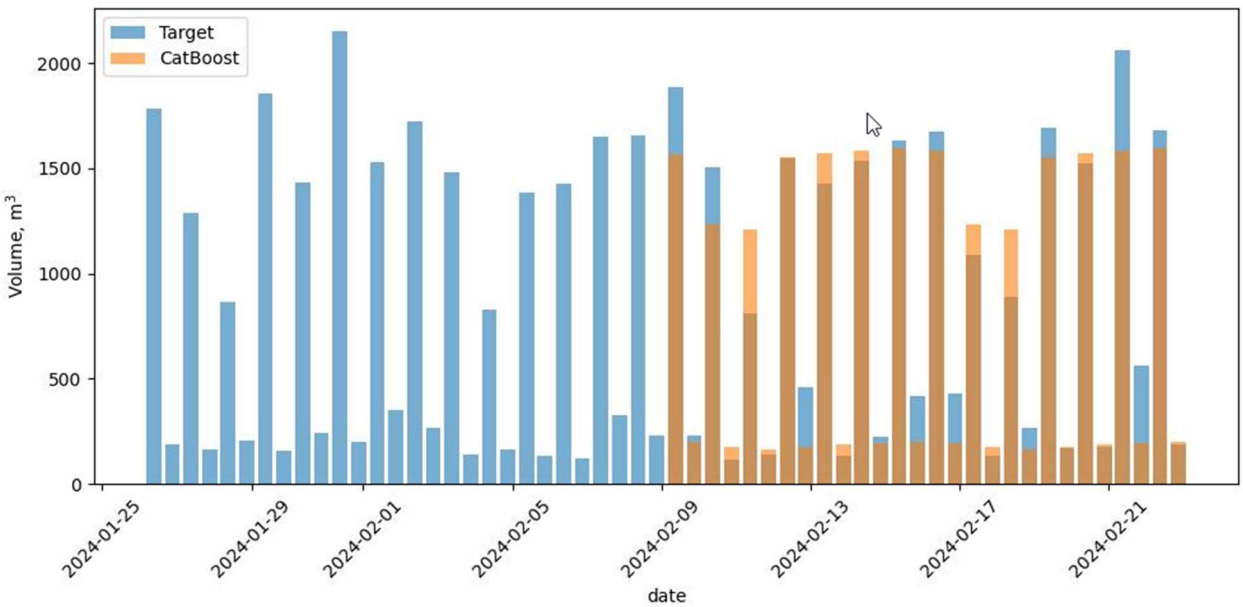

Figure 15. Predicting receipts volume for 12-hour shifts with CatBoost

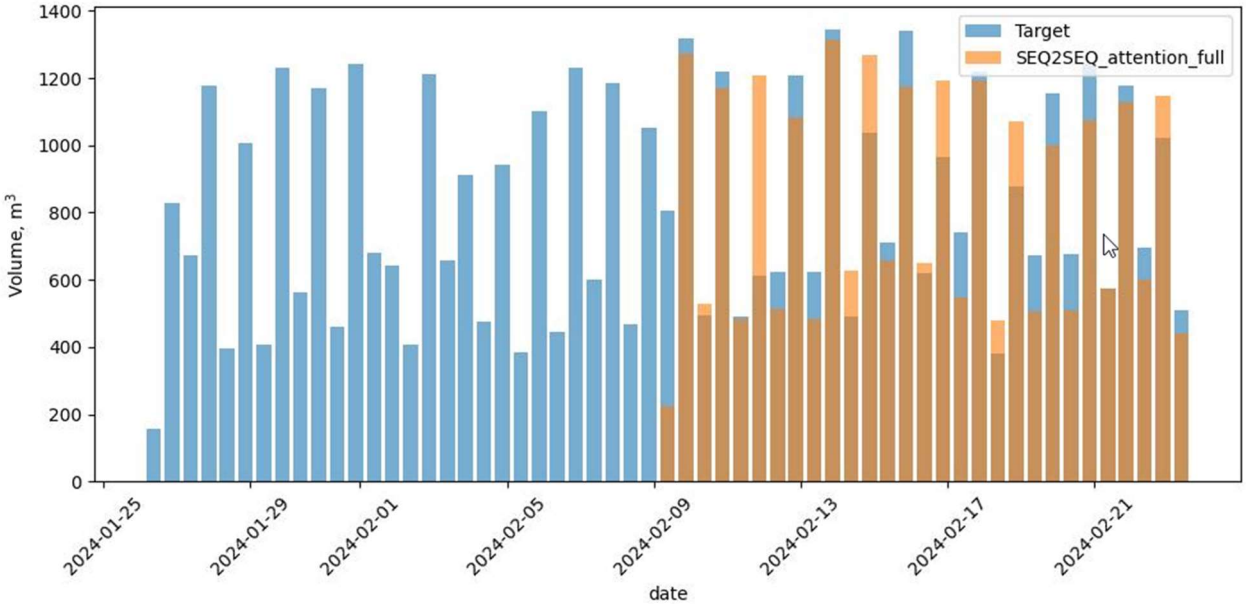

Figure 16. Predicting shipments volume for 12-hour shifts with encoder-decoder with attention

2) It was appropriate try various models and architectures as best practical results on 2 similar processes (receipts and shipments) for 2 different close prediction scale (12-hour shifts and days) were received with 4 different models.

3) Forecasting by hours within a shift is not bad, but may not be sufficient for business due to lack of reliable prediction of load peaks (See Figure 17).

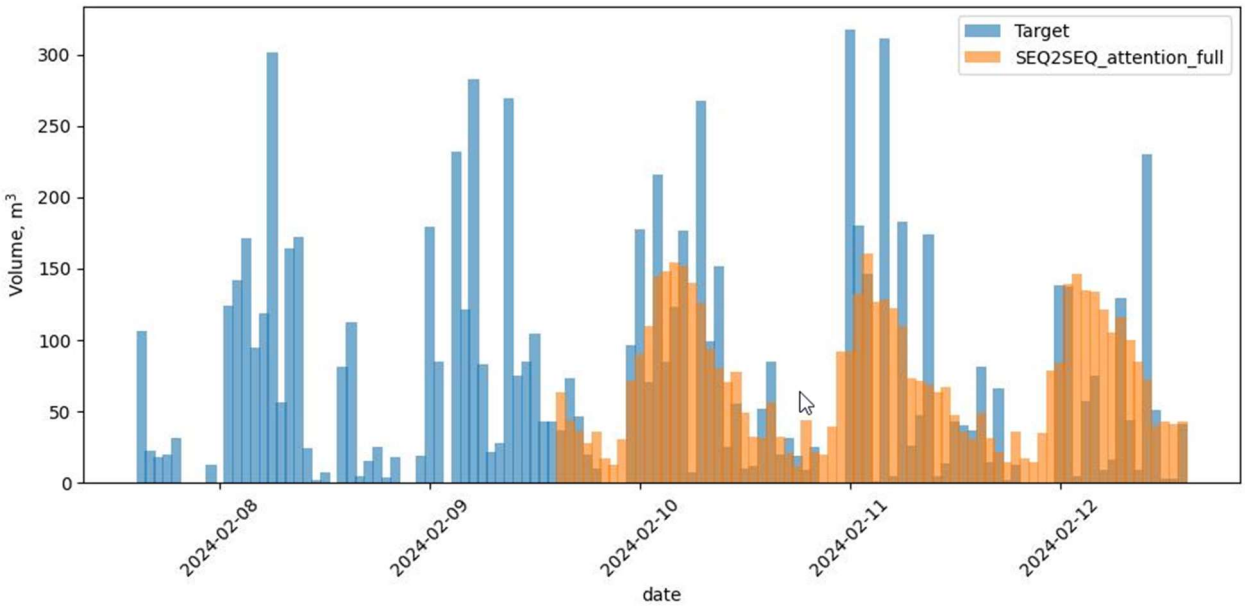

Figure 17. Predicting shipments volume for 12-hour shifts with encoder-decoder with attention

Proposals of Further Development Directions:

1) Run in MVP mode for business users to get feedback.

2) Consider the possibility of taking into account additional parameters affecting load peaks and no load to improve the accuracy of the forecast (primarily hourly)

3) Consider accounting for a longer data interval (2 years or more) to better account for long-term patterns.

There is a ready pipeline for training, deployed ready for integration server

In general, all this makes the solution maximally practical and easy to develop

REFERENCES

1) Ansari A.F., Stella L., Turkmen C., Zhang X., Mercado P., Shen H., Shchur O., Rangapuram S.S., Pineda Arango S., Kapoor S., Zschiegner J., Maddix D.C., Wang H., Mahoney M.W., Torkkola K., Wilson A.G., Bohlke-Schneider M., Wang Y. “Chronos: Learning the Language of Time Series”. arXiv preprint, arXiv:2403.07815, 2024.

2) Anubha Goel, Puneet Pasricha и Juho Kanniainen. “Time-Series Foundation AI Model for Value-at-Risk Forecasting”. arXiv preprint, arXiv:2410.11773, version 7. 2025.

3) Big Data School. Почему генеративный ИИ хорошо подходит для прогнозирования временных рядов. URL: https://bigdataschool.ru/blog/news/machine-learning/gan-for-time-series-forecasting.html (дата обр. 25.03.2025).

4) Azul Garza, Cristian Challu и Max Mergenthaler-Canseco. “TimeGPT-1”. arXiv preprint, arXiv:2310.03589, version 3. 2024

5) Aobo Liang, Xingguo Jiang и Yan Sun. "Bi-Mamba+: Bidirectional Mamba for Time Series Forecasting". arXiv preprint, arXiv:2404.15772, version 3. 2024.

6) Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y. Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan и Qingsong Wen. "Time-LLM: Time Series Forecasting by Reprogramming Large Language Models". arXiv preprint, arXiv:2310.01728, version 2. 2024.

7) Abhimanyu Das, Weihao Kong, Rajat Sen и Yichen Zhou. "A decoder-only foundation model for time-series forecasting". arXiv preprint, arXiv:2310.10688, version 4. 2024.

8) Salesforce. Moirai: Universal Time Series Forecasting. URL: https://www.salesforce.com/blog/moirai/ (дата обр. 25.03.2025).

9) Jimeng Shi, Mahek Jain и Giri Narasimhan. "Time Series Forecasting (TSF) Using Various Deep Learning Models". arXiv preprint, arXiv:2204.11115, version 1. 2022.

10) Айлогика. Создаем уникальные ИТ-решения для бизнеса. URL: https://ailogica.ru (дата обр. 26.02.2025).

11) Manhattan Associates. Labor Management System. URL: https://www.manh.com/solutions/supply-chain-management-software/labor-management-system (дата обр. 10.03.2025).

12) CDDI. 1C:WMS Логистика. Управление складом. URL: https://cddi.ru/resheniya/yslyga-avtomatizatsiya-sklada-2/ (дата обр. 10.03.2025)

13) TopLog. Система управления складом TopLog WMS. URL: https://www.toplogwms.ru/wms/ (дата обр. 10.03.2025).

14) Яндекс.Образование. Временные ряды. URL: https://education.yandex.ru/handbook/ml/article/vremennye-ryady (дата обр. 25.05.2025).

15) Ricardo P. Masini, Marcelo C. Medeiros и Eduardo F. Mendes. "Machine Learning Advances for Time Series Forecasting". arXiv preprint, arXiv:2012.12802, version 3. 2021.

16) Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush и Andrey Gulin. "CatBoost: unbiased boosting with categorical features". arXiv preprint, arXiv:1706.09516, version 5. 2019.

17) Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser и Illia Polosukhin. "Attention Is All You Need". arXiv preprint, arXiv:1706.03762, version 7. 2013.

18) Shizhe Li. Hybrid LSTM-SARIMA Model for Inflation Forecasting Based on Discrete Wavelet Transform. Proceedings of the 2nd International Conference on Financial Technology and Business Analysis. 2023.

19) Statsmodels. Statsmodels: statistical modeling and econometrics in Python. URL: https://www.statsmodels.org/stable/index.html (дата обр. 10.04.2025).

20) Scikit-learn. Scikit-learn: Machine Learning in Python. URL: https://scikit-learn.org/stable/index.html (дата обр. 15.04.2025).

21) PyTorch. PyTorch: An open source machine learning framework. URL: https://pytorch.org (дата обр. 05.05.2025).

22) FastAPI. FastAPI framework, high performance, easy to learn, fast to code, ready for production. URL: https://fastapi.tiangolo.com/ (дата обр. 20.05.2025).

23) Ключевые характеристики Axelot WMS X5.. URL: https://www.axelot.ru/product/axelot-wms-x5/ (дата обр. 10.05.2025).

APPENDIX 1

Link to GitHub Repository:

https://github.com/IvanArctos/Warehouse_prediction.git